

Transformações Geométricas

Fundamentos para Modelagem 2D/3D

Por que Transformações?

Operações essenciais em CG:

1. **Posicionamento** de objetos na cena
2. **Animações** (translação, rotação, escala)
3. **Projeções** ($3D \rightarrow 2D$)
4. **Hierarquia** de objetos (ex: braço robótico)

graph TD

A[Objeto Original] --> B[Transformação]

B --> C[Objeto Transformado]

Transformações Básicas 2D

1. Translação

- **Equação:**

$$\begin{cases} x' = x + t_x \\ y' = y + t_y \end{cases}$$

- **Matriz:**

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

Rotação 2D

Transformação angular em torno da origem

- **Equação:**

$$\left(\begin{cases} x' = x \cdot \cos\theta - y \cdot \sin\theta \\ y' = x \cdot \sin\theta + y \cdot \cos\theta \end{cases} \right)$$

- **Matriz Homogênea:**

$$\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Composição de Transformações

Ordem importa! (Não comutativa)

1. Sequência correta:

Escala $\rightarrow$ Rotação $\rightarrow$ Translação

2. Problema comum:

Rotacionar em torno de um ponto arbitrário

```
graph LR
```

```
A[Objeto] --> B[Transladar para origem]
```

```
B --> C[Rotacionar]
```

Transformações 3D

Extensão do 2D com coordenada (z)

Translação 3D

```
( \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} )
```

Escala 3D

```
( \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} )
```

Rotação 3D

Eixos principais:

- **X:** ($R_x(\theta)$)
- **Y:** ($R_y(\theta)$)
- **Z:** ($R_z(\theta)$)

Aplicações Práticas

Exemplos reais em CG

1. Hierarquia de Objetos:

```
graph TB
  A[Personagem] --> B[Braço]
  B --> C[Antebraço]
  C --> D[Mão]
```

2. Câmera Virtual:

- Sistema de coordenadas de visualização

Matriz View (rotação +

Transformações Geométricas para a Renderização Gráfica

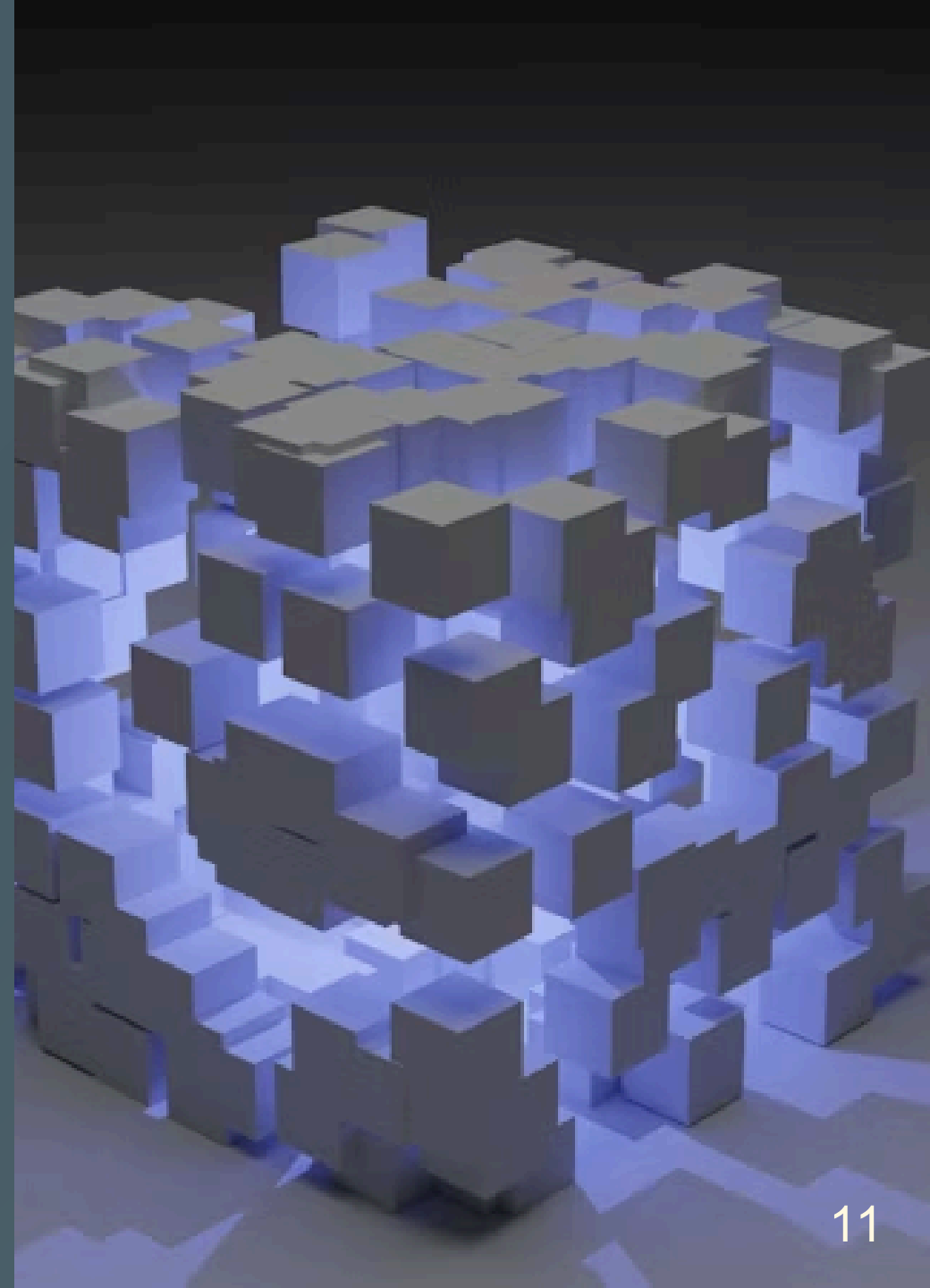


Demonstração Interativa

Explore transformações
em tempo real

Ferramentas sugeridas:

- Three.js (WebGL)
- OpenGL/WebGL Shaders
- Blender Python API



Referências

1. Foley, Van Dam. *Computer Graphics: Principles and Practice*
2. Shirley, Marschner. *Fundamentals of Computer Graphics*
3. [LearnOpenGL - Transformations](#)
4. [3Blue1Brown - Essence of Linear Algebra](#)

Imagens: Wikimedia Commons, Unsplash (licenças livres)

Recursos Visuais Incluídos:

1. Diagramas vetoriais SVG da Wikimedia
2. Ilustrações matemáticas de matrizes
3. Grafos Mermaid para hierarquias
4. Pseudocódigo para implementação
5. Links para tutoriais interativos

Como Executar:

```
marp transformacoes-geometricas.md -o transformacoes.pdf --html  
# Ou para servir localmente:  
marp -s transformacoes.md
```

Dica: Use o tema `gaia` com `invert` para melhor contraste em projetores, e adicione `<!-- _transition: slide/fade -->` entre slides para efeitos visuais suaves.